

Classifying Productions and Progressive Error Recovery

A Structural Model for Error Recovery in the Bliss Parser

Mayukh Chakraborty

Compiler Engineering

September 2026

ABSTRACT

Error recovery is often treated as an ad-hoc collection of synchronization tokens: when parsing fails, skip forward until a token from some safe set is found. This works, but it quickly becomes hard to reason about once productions are nested, recursive, optional, repeated, or branched. This paper develops an alternative foundation, organized around a single question: given that a production cannot continue normally, where can it safely resume? We introduce a structural classification of parser productions into four categories—bounded, branch, extension, and until productions—and a recovery mechanism, progressive synchronization, built on top of it. We show that these categories are compositional, that synchronization boundaries can propagate both downward from a context into a child production and upward from a child back to its caller, and that a boundary token may safely occur inside the very structure it bounds, provided the production graph guarantees the boundary remains resolvable. The result is a recovery model in which structure, rather than token identity alone, determines where and how parsing may resume.

Keywords: parsing, error recovery, synchronization, grammar productions, compiler design

1. INTRODUCTION

Error recovery changed the way this work approaches parser productions. Initially, it is tempting to think of error recovery as a collection of synchronization tokens: “if something goes wrong, skip tokens until you find one of these.” That works, but it quickly becomes difficult to reason about once productions become nested, recursive, optional, repeated, or branched.

The more useful question turned out to be: given that a production cannot continue normally, where can it safely resume? This led to a structural classification of productions and, more importantly, to a recovery mechanism based on *progressive synchronization*.

The classification used by the Bliss parser consists of four production types—bounded productions, branch productions, extension productions, and until productions. These are not intended to be a universal classification of grammars; they are an engineering model for the Bliss parser and its recovery system. Section 2 introduces progressive synchronization in isolation, Sections 3–6 work through the four production classes in turn, Section 7 generalizes the notion of a boundary to productions themselves, and Sections 8–9 draw the pieces together into a single principle.

2. PROGRESSIVE SYNCHRONIZATION

Consider a simple production $A \rightarrow BCD$, and suppose B is missing. Rather than immediately abandoning the production, we can use C as evidence that B was omitted, treating $B \rightarrow C$ as a fallback. If C is missing as well, D can become the next synchronization point, giving $C \rightarrow D$. Eventually

the production reaches the last boundary it owns, and at that point recovery becomes the responsibility of the surrounding production. This produces a progressive chain:

$$B \rightarrow C \rightarrow D \rightarrow \text{external synchronization.}$$

The important point is that a synchronization token is not merely a token that is globally considered “safe.” It is a token that represents a *later structural state*. This is the foundation on which all four production types below are built.

3. BOUNDED PRODUCTIONS

A bounded production has a definite structural beginning and end. For example, *function* ::= “fx” *identifier* “(” “)” “{” “}” has a structure known before parsing begins: $\text{fx} \rightarrow \text{identifier} \rightarrow (\rightarrow) \rightarrow \{ \rightarrow \}$. This gives a natural progression of synchronization points:

```
function parseFunction(parser, syncTokens) {  
  match("fx");  
  match(parser.identifier, "(");  
  match("(", ")");  
  match(")", "{");  
  match("{", "}");  
  match("}", syncTokens);  
}
```

The first token needs no synchronization once the production has already been selected: if the parser has decided that this is a function production, the *fx* token establishing that production already exists, and recovery begins with the subsequent structure—*identifier* $\rightarrow (, (\rightarrow),) \rightarrow \{, \{ \rightarrow \}$, and finally $\} \rightarrow \text{external synchronization}$.

The final token is special. Once `}` has been reached, the bounded production has exhausted the structure it owns, and the question of what comes next belongs to the caller. This is why bounded productions take an external synchronization set as a parameter, as in `parseFunction(parser, syncTokens)`.

Principle 1. *A bounded production can progressively synchronize through its known internal boundaries. Once its final boundary is reached, further synchronization is external.*

4. BRANCH PRODUCTIONS

A branch production chooses one of several alternatives, as in `program ::= function | variable`. The branches are not independent structures; they converge back into a common continuation. Consider `var a : type = b;` with a branch such as `var a (":" "type" "=" "b" | "=" "b") ";"`. Both arms converge at `;`, so `;` is the synchronization boundary for each arm.

Rather than manually supplying that boundary to every branch—writing `parseArmA(sync = ";")` and `parseArmB(sync = ";")` separately—the branch can establish the context once:

```
useBranch([parseA, parseB, parseC], ";");
```

This propagates the convergence boundary into the selected alternative. It is not merely a dispatch abstraction; it establishes a recovery context in which each candidate is entered with `sync = continuation` already attached.

Principle 2. *A branch production propagates its convergence boundary into its alternatives, allowing an incomplete alternative to recover where all alternatives rejoin.*

The branch itself is also responsible for choosing between possible production starts. Once an alternative has been selected, recovery belongs to that selected production.

5. EXTENSION PRODUCTIONS

An extension production represents an optional continuation of an already valid structure. For example, `a b` and `a b c` can be interpreted as a base production `a b` followed by an optional extension `c d`.

Principle 3. *If the extension cannot be entered, there is no error.*

For `a b`, the absence of `c` simply means that the base production is complete—there is no such thing as a “missing extension.” The control flow is essentially: if the extension’s start token exists, enter the extension; if it is absent, the base production is already complete.

However, once the extension’s starting condition is encountered, the parser has committed to the extension. In `a b c`, where the extension is `c d`, the token `c` establishes

the existence of the extension, and `d` is now required. So `a b` is valid, while `a b c` contains an error if `d` is missing. The extension itself needs no special recovery strategy of its own—its internal productions take responsibility once the extension has been entered.

Principle 4. *Before commitment, absence of an extension is valid. After commitment, errors belong to the productions inside the extension.*

Extensions can naturally contain other production types (an extension containing a branch, which in turn contains a bounded or until production, and so on), so recovery is compositional from the start.

6. UNTIL PRODUCTIONS

An until production represents repetition terminated by an end boundary, as in `program ::= {function | variable}`, or more concretely `until(production, separator, end)`. The until construct does not own the internal structure of the repeated production—it owns the mechanics of repetition: the separator, if one exists, and the termination boundary.

Consider `int a, int b, int c)` parsed with `until(variable, sep = ",", end = ")")`. After parsing a production, there are three basic cases. If the end token exists, the repeated region is complete and no separator is required ($P \rightarrow \text{END}$). If the end token does not exist but the separator does, the separator proves that another production follows ($P \rightarrow \text{separator} \rightarrow P$). If neither exists, the until production synchronizes toward its end boundary.

Principle 5. *The separator primarily controls continuation. The end token is the recovery boundary owned by the until production.*

6.1 Dynamic linearization

Although an until production is conceptually unbounded, every concrete, successfully terminated input is finite. For example, `P, P, P, P;` can be viewed as $P_1, P_2, P_3, P_4;$. Once the scope is known, the repeated structure has effectively become a finite sequence; conceptually, `linearize(until, scope)` turns a repetition into a dynamic bounded structure, which allows until recovery to be reasoned about with the same progressive-boundary ideas as bounded productions.

Two forms are worth distinguishing. With an *end-exclusive* separator, as in `P, P, ..., P;`, the separator occurs strictly between productions and the final production ends directly at the delimiter. With an *end-inclusive* separator, as in `P, P, ..., P, ;`, the final separator is part of the repeated unit, and the grammar can be normalized as $\{ P \text{ " , " } \} \text{END}$ rather than $P \{ \text{ " , " } P \} \text{END}$.

Principle 6. *An end-inclusive separator is effectively a separator embedded inside the repeated production unit.*

7. PRODUCTION AS A SYNCHRONIZATION BOUNDARY

A token is not the only thing that can provide a synchronization boundary—a production itself can establish a boundary for another production. This matters for recursive structures such as expressions. Consider

$$\begin{aligned} atom &\rightarrow "[" \text{ expr } ";" \text{ expr } "]" , \\ sum &\rightarrow atom \text{ ("+" atom) } ^? , \\ ops &\rightarrow sum \text{ ("*" sum) } ^? , \\ expr &\rightarrow ops , \end{aligned}$$

with the outer context `var name = expr ;`. The outer production can provide `expr(sync = ";" )`, and the boundary propagates through the nested productions from `expr` to `ops` to `sum` to `atom`. The same synchronization boundary can therefore travel through an arbitrary number of production layers: a production does not need to know its absolute end in isolation—it only needs a boundary supplied by its surrounding context.

7.1 Start-of-file and end-of-file

This naturally leads to a global invariant. Introduce *SOF* as the start-of-file boundary; the entire production graph then has the form $SOF \rightarrow program \rightarrow \dots \rightarrow EOF$. There are no hanging production threads: every production is ultimately situated inside some surrounding structural context, whether that means an intrinsic boundary, a boundary inherited from its caller, or a boundary established by another nested production.

7.2 Boundary tokens can be shared

A synchronization token does *not* have to be outside the production that receives it. The same token can be part of the internal grammar, the boundary of the surrounding production, and the synchronization point for recovery, all at once. Consider $atom \rightarrow number \mid text$, $sum \rightarrow atom \text{ ("+" atom) } ^?$, $expr \rightarrow sum$, with the deliberately strange surrounding rule `var name = expr "+"`. Here `+` is both part of the expression grammar and the external boundary of `expr`. In `x + 10 + 20 ++`, the expression is malformed, but the final `+` still provides a recoverable boundary. In `x + 10 + 20`, the expression resolves, but the surrounding production is missing its required `+`. Both inputs contain errors, but both remain structurally recoverable: there is no requirement that the parser reconstruct the user's exact intention, only that it find a structurally viable continuation.

7.3 Boundary resolvability

This gives a stronger rule than simply forbidding synchronization tokens from occurring inside an expression—that rule would be unnecessarily restrictive.

Principle 7. *A boundary token may occur inside a nested or unbounded context, provided the production graph guarantees that the boundary can eventually be resolved.*

Production	Structural property	Recovery responsibility
Bounded	Known internal start and end	Progressive synchronization through known boundaries; external beyond the final one
Branch	Alternatives converging to one continuation	Propagate the convergence boundary into the selected alternative
Extension	Optional continuation of a base structure	No error before commitment; internal productions own errors after
Until	Repetition with a termination boundary	Control repetition; end boundary for recovery, separator for continuation

TABLE 1. Structural responsibilities of the four production classes.

For example, `(a + b)` can safely use `+` internally even if `+` is also a surrounding synchronization token, because the parenthesized structure establishes its own boundary: the expression resolves before the closing `)` is returned to the enclosing production. The issue is therefore not token containment but whether the grammar can create a path that consumes the boundary indefinitely without ever returning control to its surrounding context. The relevant property is *boundary resolvability*: a production may contain its surrounding boundary token, provided the production graph still offers a structural path by which the boundary becomes observable and control can return to the caller.

8. THE UNIFIED MODEL

Table 1 summarizes the structural responsibilities of the four production classes.

These classifications are *compositional*: a production can contain another production, as in an `until` that contains a branch, which contains an extension, which contains a bounded production. The parser does not need a distinct recovery mechanism for every combination—recovery information simply flows through the production graph.

9. THE CENTRAL PRINCIPLE

The entire model can ultimately be reduced to one idea.

Principle 8. *Recovery follows structure rather than merely tokens.*

A synchronization token is useful because it represents a structurally meaningful future state: a branch contributes its convergence point, a bounded production contributes its known internal boundaries, an extension contributes a commitment point, and an `until` production contributes a termination boundary together with its continuation mechanism. An enclosing production, in turn, can provide a

synchronization boundary to an otherwise open-ended child. Information flow is therefore both downward and upward: a synchronization boundary flows down from context into the child production; when the child cannot recover internally, responsibility flows back up to the parent.

The parser does not need to guess the entire intended program after an error. It only needs to preserve enough structure to continue parsing the remainder of the input. That is the purpose of progressive synchronization.

10. SUMMARY

This paper presents a structural model for parser error recovery based on progressive synchronization. The model

classifies productions as bounded, branch, extension, and until productions, with each class contributing a specific form of structural recovery. Synchronization boundaries can propagate downward from a surrounding context into nested productions and responsibility can propagate upward when a child production can no longer recover internally. The central idea is that recovery should follow the structure of the grammar rather than merely the identity of individual tokens. A boundary is therefore useful when it represents a structurally viable point at which parsing can resume, even when that boundary token also appears inside the structure being recovered.